

[NPM3D] DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation

Rayane Dakhlaoui
rayane.dakhlaoui@telecom-paris.fr
Télécom Paris - PSL University
Paris, France

Abstract

This report, prepared for the 3D Point Cloud and Modeling (NPM3D) course of the IASD Master’s program, presents an in-depth study of *DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation* by Park et al. The authors propose DeepSDF, a learned continuous Signed Distance Function (SDF) representation that encodes an entire class of 3D shapes within a compact latent space. Rather than relying on discrete volumetric grids or parametric meshes, DeepSDF represents surfaces implicitly as the zero-level-set of a neural network conditioned on a latent code. The architecture introduces a probabilistic *auto-decoder* formulation that consists of training a decoder-only network by jointly optimizing both the network weights and per-shape latent vectors, which naturally supports inference from partial observations through MAP estimation. This study reviews the theoretical foundations of the approach, analyzes its architecture and training procedure, and presents reproduced experiments on shape reconstruction and latent space interpolation, confirming the paper’s central claims.

Project Resources:

- **Official codebase:** Facebook Research - Deep SDF
- **Implementation used:** maurock-DeepSDF
- **Original paper:** arXiv:1901.05103
- **Code & Checkpoints:** Google Drive
- **Dataset:** ShapeNet

1 Introduction

Reconstructing and representing 3D geometry is a fundamental problem in computer vision, graphics, and robotics. Three main families of representations have dominated data-driven 3D learning: point clouds, meshes, and voxels. Each comes with inherent limitations. Point clouds are memory-efficient and naturally match sensor outputs (LiDAR, depth cameras), but they do not encode surface topology and cannot directly produce watertight surfaces. Mesh-based methods struggle with arbitrary topologies and typically require pre-defined templates. Voxel grids offer a regular structure amenable to 3D convolutions, but their memory cost grows cubically with resolution, limiting practical resolutions.

An alternative family, implicit surface representations, defines a surface as the zero-level-set of a continuous scalar function:

$$S = \{\mathbf{x} \in \mathbb{R}^3, f(\mathbf{x}) = 0\}. \quad (1)$$

Classical implicit methods such as Implicit Moving Least Squares (IMLS) and Screened Poisson Reconstruction (SPR) define this function locally or globally from oriented point clouds. These methods reconstruct a single shape and do not generalize across a dataset.

Park et al. propose DeepSDF, which learns a continuous SDF for an entire class of shapes using a deep feed-forward network

conditioned on a per-shape latent code. The surface is implicitly encoded as the decision boundary of the network, enabling high-quality shape reconstruction, smooth interpolation in latent space, and flexible completion from partial observations.

The three main contributions of the paper are:

- (1) A formulation of generative 3D shape modeling with a continuous implicit surface conditioned on a latent code.
- (2) An *auto-decoder* learning scheme that jointly optimizes network weights and per-shape latent vectors without an encoder, with a probabilistic MAP inference procedure at test time.
- (3) State-of-the-art results on shape reconstruction and completion, with a model size an order of magnitude smaller than prior work.

2 Mathematical Foundations

2.1 Signed Distance Functions

A **Signed Distance Function** is a continuous scalar field that, for any query point $\mathbf{x} \in \mathbb{R}^3$, returns its signed distance to the nearest surface:

$$\text{SDF}(\mathbf{x}) = s \in \mathbb{R}, \quad (2)$$

where the sign encodes whether the point lies inside ($s < 0$) or outside ($s > 0$) the shape, and the magnitude gives the distance to the surface boundary. The surface itself is implicitly represented as the zero-level-set $\text{SDF}(\mathbf{x}) = 0$.

Given a learned or computed SDF, the corresponding mesh can be extracted by evaluating the field on a regular grid and applying Marching Cubes, an algorithm that classifies each voxel cell as interior, exterior, or intersecting, and generates 0, 1, 2, or 3 triangles per cell from a lookup table of $2^8 = 256$ configurations. This provides a seamless bridge between the implicit neural representation and a standard triangular mesh.

Classical SDF approaches discretize the field onto a voxel grid. Fusion methods such as KinectFusion accumulate noisy depth maps into a Truncated SDF (TSDF) at fixed resolution. DeepSDF instead represents the SDF as a *continuous learned function*, avoiding resolution constraints entirely.

2.2 Single-Shape DeepSDF

The simplest instantiation of DeepSDF trains a single network $f_\theta : \mathbb{R}^3 \rightarrow \mathbb{R}$ to approximate the SDF of one target shape. Given a training set of point-SDF pairs:

$$X = \{(\mathbf{x}_j, s_j) : s_j = \text{SDF}(\mathbf{x}_j)\}, \quad (3)$$

the network is trained by minimizing a clamped L_1 loss:

$$\mathcal{L}(f_\theta(\mathbf{x}), s) = |\text{clamp}(f_\theta(\mathbf{x}), \delta) - \text{clamp}(s, \delta)|, \quad (4)$$

where $\text{clamp}(x, \delta) = \min(\delta, \max(-\delta, x))$. The hyperparameter δ controls the extent of the surface over which a metric SDF is enforced: - Small δ concentrates network capacity near the surface to capture fine geometric detail - Large δ maintains a wider safe zone useful for fast ray-casting, where each sample provides a guaranteed step size.

2.3 Latent Space and Coded Shape DeepSDF

To represent an entire class of shapes, a latent code $\mathbf{z} \in \mathbb{R}^L$ is introduced as a second input to the network. The network $f_\theta(\mathbf{z}_i, \mathbf{x})$ now approximates the SDF of the i -th shape:

$$f_\theta(\mathbf{z}_i, \mathbf{x}) \approx \text{SDF}^i(\mathbf{x}). \quad (5)$$

Conceptually, the latent code $\mathbf{z}_i$ encodes the geometric identity of the shape, and the network acts as a universal shape decoder. Different values of $\mathbf{z}$ produce different shape SDFs, and the continuous nature of the latent space enables smooth interpolation between shapes.

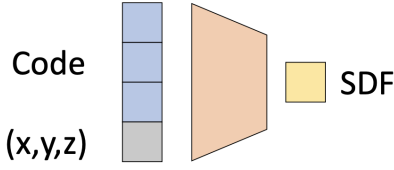


Figure 1: Coded Shape DeepSDF: the input to the decoder is the concatenation of a per-shape latent code $\mathbf{z}_i$ (blue) and the 3D query point (x, y, z) (gray). The network maps this to a single scalar SDF value.

2.4 Auto-Decoder Formulation

Motivation. Auto-encoders are the standard tool for learning latent representations: an encoder maps inputs to codes, and a decoder reconstructs outputs. However, Park et al. observe that in recent 3D learning works, the encoder is only used during training, discarded at inference, and replaced by optimization-based code search. This raises the question: why train an encoder at all? The auto-decoder skips the encoder entirely and directly optimizes the latent codes alongside the decoder weights during training.

Probabilistic derivation. Given N training shapes, with shape i represented by SDF samples $\mathcal{X}_i = \{(\mathbf{x}_j, s_j)\}$, the posterior over the latent code $\mathbf{z}_i$ given the observations factorizes under conditional independence as:

$$p_\theta(\mathbf{z}_i | \mathcal{X}_i) = p(\mathbf{z}_i) \prod_{(\mathbf{x}_j, s_j) \in \mathcal{X}_i} p_\theta(s_j | \mathbf{z}_i; \mathbf{x}_j). \quad (6)$$

A zero-mean isotropic Gaussian prior is placed on the latent codes: $p(\mathbf{z}_i) = \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$. The SDF likelihood is expressed via the network prediction:

$$p_\theta(s_j | \mathbf{z}_i; \mathbf{x}_j) = \exp(-\mathcal{L}(f_\theta(\mathbf{z}_i, \mathbf{x}_j), s_j)). \quad (7)$$

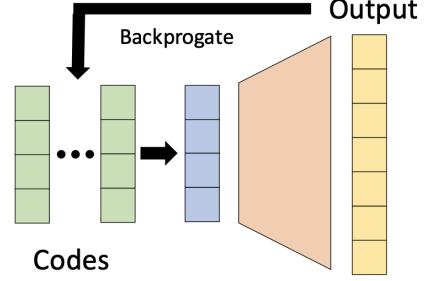


Figure 2: Auto-decoder training scheme. The per-shape latent codes (green) are not produced by an encoder but are instead optimized directly via backpropagation alongside the network weights θ . At each iteration, the gradient flows from the reconstruction loss back through the decoder into the latent codes.

Training objective. Maximizing the joint log-posterior over all training shapes with respect to both θ and $\{\mathbf{z}_i\}_{i=1}^N$ yields:

$$\arg \min_{\theta, \{\mathbf{z}_i\}} \sum_{i=1}^N \left(\sum_{j=1}^K \mathcal{L}(f_\theta(\mathbf{z}_i, \mathbf{x}_j), s_j) + \frac{1}{\sigma^2} \|\mathbf{z}_i\|_2^2 \right). \quad (8)$$

The regularization term $\frac{1}{\sigma^2} \|\mathbf{z}_i\|_2^2$ prevents the latent codes from diverging and encourages a compact, well-structured latent manifold.

Inference (MAP estimation). At test time, the decoder weights θ are frozen and the optimal latent code for a new shape (observed through an arbitrary set of SDF samples $\mathcal{X}$) is recovered by solving:

$$\hat{\mathbf{z}} = \arg \min_{\mathbf{z}} \sum_{(\mathbf{x}_j, s_j) \in \mathcal{X}} \mathcal{L}(f_\theta(\mathbf{z}, \mathbf{x}_j), s_j) + \frac{1}{\sigma^2} \|\mathbf{z}\|_2^2. \quad (9)$$

This formulation is a critical advantage over encoder-based approaches: since the gradient of the loss with respect to $\mathbf{z}$ can be computed independently for each sample, inference works with *any number and distribution of SDF observations*, including partial depth maps or noisy point clouds. In contrast, an encoder requires inputs of the same format as training data.

Comparison with VAE and standard AE. The authors validate this design on MNIST using 2D, 5D, and 15D latent codes. The auto-decoder produces smooth latent spaces comparable to the Variational Auto-Encoder (VAE), and its reconstructions more closely match the actual test digits. The standard auto-encoder, when used with optimization-based decoding at test time, fails entirely, its latent space contains many bad local minima unsuited for gradient-based code search.

3 Architecture and Training Pipeline

3.1 Network Architecture

The DeepSDF network is a fully-connected feed-forward network with the following design choices, each motivated by empirical findings reported in the paper:

Depth and skip connections. The network consists of 8 fully-connected layers of dimension 512, with ReLU activations and a

dropout rate of 0.2. A skip connection is inserted at the 4th layer: the original input vector ($z_i, \mathbf{x}$) is concatenated with the intermediate feature vector, forming a 512-dimensional vector that feeds into the 5th layer. Without skip connections, the training loss saturates after 4 layers. With them, the loss continues to decrease up to 16 layers, with 8 layers offering a good speed-accuracy trade-off.

Normalization. Batch normalization was found to be *unstable* in this setting and was replaced by weight normalization, which reparameterizes each weight vector by its norm and direction without introducing batch statistics.

Input and output. The input to the network is the concatenation of the latent code $\mathbf{z}_i \in \mathbb{R}^L$ and the 3D query point $\mathbf{x} \in \mathbb{R}^3$, forming a vector of size $L + 3$ (259 for $L = 256$). The output is a single scalar $\hat{s} \in [-1, 1]$, produced by a tanh activation on the final layer.

Latent vector size. The latent dimension is set to $L = 256$ for shape reconstruction and $L = 128$ for shape completion. Latent vectors are initialized from $\mathcal{N}(0, 0.01^2)$ to keep all shape codes near the origin at the start of training.

3.2 Data Preparation

Each training shape from ShapeNet is normalized to a unit sphere. SDF training samples are generated as follows:

- (1) **Near-surface sampling:** 250,000 points are sampled on the mesh surface, weighted by triangle area. Each surface point is perturbed twice along all axes with zero-mean Gaussian noise of variance 0.0025 and 0.00025, producing two spatial samples per surface point. This aggressive near-surface strategy is key to capturing fine geometric detail near the zero-level-set.
- (2) **Uniform sampling:** 25,000 points are sampled uniformly inside the unit sphere to provide global SDF supervision.
- (3) **Sign computation:** For each query point, the nearest surface point is found using a KD-tree. The sign is determined from the dot product between the surface normal and the vector from the surface point to the query.
- (4) **Watertight filtering:** Meshes with more than 2% double-sided triangles (indicating a non-watertight surface) are discarded, as their SDF sign is ill-defined.

3.3 Training Details

In the paper, Adam optimizer is used throughout. Learning rates are $10^{-4} \times B$ for the decoder parameters (where B is the batch size) and 10^{-3} for the latent vectors. The regularization parameter is set to $\sigma = 10^{-2}$. For each shape in a batch, 16,384 SDF samples are drawn with a balanced 50/50 split between positive (outside) and negative (inside) samples. Training runs for 1,000 epochs on 8 NVIDIA GPUs for approximately 8 hours.

The truncation parameter δ is set to 0.1 for all main experiments. The authors report that increasing δ moderately reduces surface accuracy (measured by Chamfer Distance) while providing a larger safe zone for ray-casting leading to a trade-off between rendering speed and geometric precision.

4 Experiments

The paper evaluates DeepSDF on four tasks using the ShapeNet dataset: representing known training shapes (K), representing unseen test shapes (U), completing partial shapes (C), and latent space interpolation. The baselines are OGN (octree-based voxel), AtlasNet (parametric mesh), and 3D-EPN (volumetric SDF completion).

Evaluation metrics include:

- **Chamfer Distance (CD):** Sum of nearest-neighbor distances between two point sets, computed with 30,000 points.
- **Earth Mover’s Distance (EMD):** Optimal bijection-based distance, computed with 500 points.
- **Mesh Accuracy:** Minimum distance d such that 90% of generated points lie within d of the ground truth mesh.
- **Mesh Completion:** Fraction of ground truth points within $\Delta = 0.01$ of the generated mesh.
- **Cosine Similarity:** Mean cosine similarity between normals of nearest point pairs.

4.1 Quantitative Results from the Paper

Table 1 reports the results for representing known (training) shapes on the car category. DeepSDF achieves significantly lower Chamfer Distance than all baselines while matching OGN on EMD.

Table 1: Comparison for representing known shapes (K) on ShapeNet cars. CD multiplied by 10^3 (30k points). Lower is better.

Method	CD mean	CD median	EMD mean	EMD median
OGN	0.167	0.127	0.043	0.042
AtlasNet-Sph.	0.210	0.185	0.046	0.045
AtlasNet-25	0.157	0.140	0.060	0.060
DeepSDF	0.084	0.058	0.043	0.042

Table 2 shows results for unseen shapes across multiple ShapeNet categories. DeepSDF achieves the best mean and median Chamfer Distance overall, and excels particularly on topologically complex shapes (chairs with holes) where AtlasNet struggles due to its parametric surface patches.

Table 2: Comparison for representing unknown shapes (U) on ShapeNet. CD mean multiplied by 10^3 . Lower is better.

Method	Chair	Plane	Table	Lamp	Sofa
AtlasNet-Sph.	0.752	0.188	0.725	2.381	0.445
AtlasNet-25	0.368	0.216	0.328	1.182	0.411
DeepSDF	0.204	0.143	0.553	0.832	0.132

4.2 Reproduced Experiments

4.2.1 Experimental Setup.

Implementation choice. The official Facebook Research repository for DeepSDF relies on a C++ preprocessing pipeline that must be compiled from source: the PreprocessMesh and SampleVisibleMesh-Surface binaries handle SDF computation and surface sampling. In practice, building this pipeline against modern CUDA toolchains (CUDA 11+) and current system libraries proved unreliable due to CMake dependency conflicts and ABI incompatibilities introduced since the original 2019 release. In addition, the pipeline requires the full ShapeNet archive and a specific directory structure that is non-trivial to reconstruct.

I therefore adopted a Python-only reimplementaion available on GitHub DeepSDF-maurock, which replaces the C++ preprocessing with the point-cloud-utils library for SDF computation and offers a straightforward Python installation. This implementation faithfully replicates the auto-decoder architecture, the data sampling strategy, and the training objective described in the paper.

I train on 3 ShapeNet objects from distinct categories: a bottle (cat. 02876657), a camera (cat. 02942699), and a mug (cat. 03797390). While the original paper trains on hundreds of shapes per category, this reduced setup allows us to reproduce the key qualitative behaviors of the method within the computational constraints of a single NVIDIA T4 GPU.

The data preparation follows the paper exactly: 250,000 near-surface points perturbed with Gaussian noise ($\sigma = 0.0025$ and $\sigma = 0.00025$) and 25,000 uniform points in $[-1, 1]^3$, for a total of 275,000 SDF samples per shape. The network uses 8 fully-connected layers with inner_dim= 512, a latent code of dimension $L = 64$, weight normalization, and a skip connection at layer 4. Training runs for 500 epochs with $\delta = 0.1$, Adam optimizer, $lr_{\text{model}} = 10^{-4}$, $lr_{\text{latent}} = 10^{-3}$, and $\sigma_{\text{reg}} = 10^{-2}$.

4.2.2 Experiment 1 — Shape Reconstruction from Latent Codes. Figure 3 shows the training and validation loss curves over 500 epochs. The loss decreases from approximately 10^{-2} to 7×10^{-4} , with train and validation curves nearly overlapping throughout. The absence of a generalization gap is expected in this setting: because training directly optimizes per-shape latent codes alongside network weights, the model is evaluated on the same shapes it was trained on. This is the standard auto-decoder evaluation protocol for representing known shapes (the “K” setting of the paper).

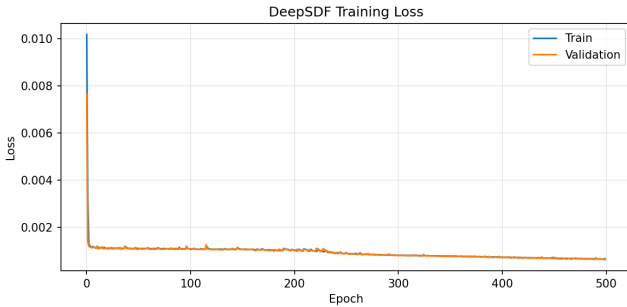


Figure 3: Training and validation loss (L1 + latent regularization) over 500 epochs. Convergence is smooth after an initial sharp drop in the first 10 epochs, with the scheduler reducing learning rates around epoch 150 and 350.

Figure 4 shows the 3D meshes obtained by evaluating the learned SDF on a 128^3 grid and extracting the zero-level-set with Marching Cubes. All three objects are clearly recognizable. The bottle body and neck are faithfully captured, including the characteristic narrowing of the neck. The camera body and lens barrel are distinctly visible, with surface detail in the lens housing region. The mug shows a smooth bowl shape with a handle correctly attached to the outer surface. These results confirm that the network has learned to associate distinct latent codes with geometrically meaningful shape representations.

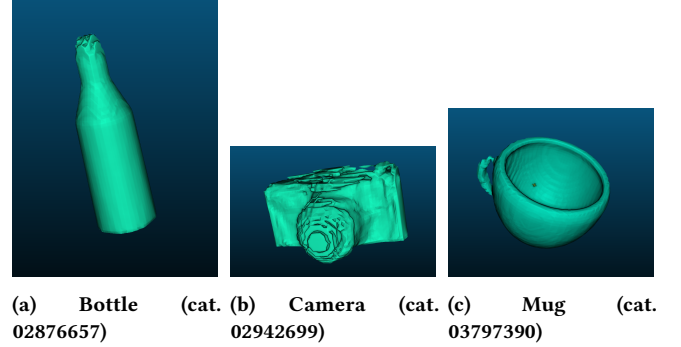


Figure 4: Meshes reconstructed from trained latent codes via Marching Cubes at resolution 128^3 . All three objects are clearly identifiable, validating that the network has learned a meaningful per-shape SDF.

Figure 5 shows 2D cross-sections of the learned SDF for each object on three orthogonal planes ($x = 0$, $y = 0$, $z = 0$). The color encodes the SDF value: blue for negative (interior), red for positive (exterior), with the zero-level-set highlighted by a black contour. Several observations are noteworthy.

First, the field is smooth and continuous everywhere in the domain, confirming that the network has learned a well-behaved scalar field rather than a lookup table. Second, the gradient magnitude near the surface is consistent with a metric SDF: colors transition sharply from blue to red within a narrow band around the zero contour. Third, the mug cross-section (Figure 5, bottom row) is particularly instructive: the handle creates a topological genus-1 structure that the SDF captures without any explicit topology constraint. This is a key advantage over mesh-based parametric approaches such as AtlasNet, which require topological templates.

4.2.3 Experiment 2 — Latent Space Interpolation. Figure 6 shows the result of linearly interpolating between the bottle ($\alpha = 0$) and camera ($\alpha = 1$) latent codes:

$$\mathbf{z}(\alpha) = (1 - \alpha) \mathbf{z}_{\text{bottle}} + \alpha \mathbf{z}_{\text{camera}}, \quad \alpha \in [0, 1]. \quad (10)$$

For each of 7 uniformly spaced values of α , I evaluate the network on a 128^3 grid and extract the corresponding mesh. The intermediate shapes form a geometrically continuous transition: the elongated bottle shape progressively shortens and widens into the rectangular camera body. Crucially, every intermediate value of α produces a valid, watertight closed surface, a property that follows from the continuity of the learned SDF as a function of $\mathbf{z}$.

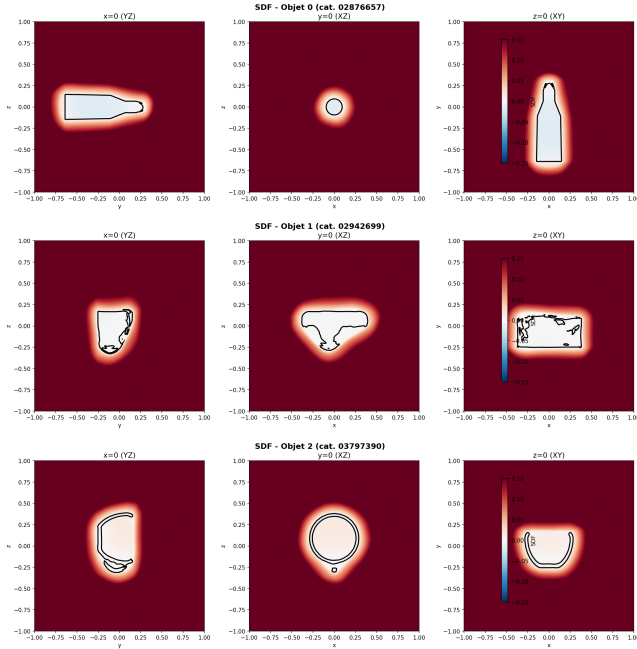


Figure 5: 2D cross-sections of the learned SDF on the three orthogonal planes for each object (top: bottle, middle: camera, bottom: mug). Blue = interior (SDF < 0), red = exterior (SDF > 0), black contour = surface (SDF = 0). The field is continuous and smooth, and the zero-level contour faithfully reproduces the object silhouette in each plane.

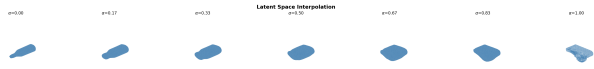


Figure 6: Linear interpolation in latent space between the bottle ($\alpha = 0$) and the camera ($\alpha = 1$), sampled at 7 values of α . The transition is geometrically continuous, and all intermediate shapes are valid watertight meshes. This reproduces the qualitative behavior of Figure 1 of the paper.

This result reproduces the qualitative behavior of Figure 1 of the paper, and provides direct empirical support for the claim that the latent space is *smoothly structured*. The regularity of intermediate shapes is not guaranteed by construction: it emerges from the Gaussian prior $p(z) = \mathcal{N}(0, \sigma^2 \mathbf{I})$, which discourages large latent codes and encourages the learned manifold to be well-conditioned around the origin.

A notable difference from the paper’s setting is that our interpolation is performed *across categories* (bottle to camera), whereas the paper interpolates within a single category (e.g., cars). Despite this harder setup, the interpolation remains geometrically coherent, suggesting that the auto-decoder framework learns a representation that is continuous with respect to geometric similarity rather than semantic class membership.

4.2.4 Experiment 3 — Ablation Study. To investigate the impact of two core architectural choices, the skip connection and the latent code dimension, I trained four model variants on the same three-object dataset for 200 epochs each, keeping all other hyperparameters fixed ($\text{lr}_{\text{model}} = 5 \times 10^{-4}$, $\sigma = 0.01$, batch size 16384). Reconstruction quality is measured by mean Chamfer Distance (CD) over the three shapes.

Table 3: Ablation results: mean Chamfer Distance over 3 shapes (lower is better).

Config	Skip	L	CD ↓
Baseline	✓	64	0.00452
No skip connection	✗	64	0.00361
Latent 128	✓	128	0.00207
Latent 256	✓	256	0.00417

Effect of latent dimension. The results in Table 3 show a clear optimum at $L = 128$, which yields the lowest mean CD (0.00207, a 54% improvement over the $L = 64$ baseline). Increasing the dimension to $L = 256$ degrades performance (CD 0.00417), which is consistent with overfitting: with only three training shapes, the higher-dimensional latent space provides excess capacity that the model cannot generalize from. This finding is aligned with the original paper’s supplementary analysis, which identifies 256 as the optimal dimension for larger datasets with hundreds of shapes, here 128 appears to be the appropriate capacity for the available data volume.

Effect of skip connections. The skip connection ablation yields an ambiguous result at this scale: the “no skip” variant achieves a lower mean CD (0.00361 vs. 0.00452). However, inspection of per-object CDs reveals that this aggregate is misleading. On the camera shape, the baseline with skip achieves CD 0.00186 compared to 0.00613 without, a factor of 3 difference in favour of the skip. The mean is dominated by the mug, for which the no-skip variant happens to perform better. With only three test shapes, per-object variance entirely swamps the systematic effect of the skip connection. The paper’s original evaluation over hundreds of shapes per category is necessary to reliably measure this contribution.

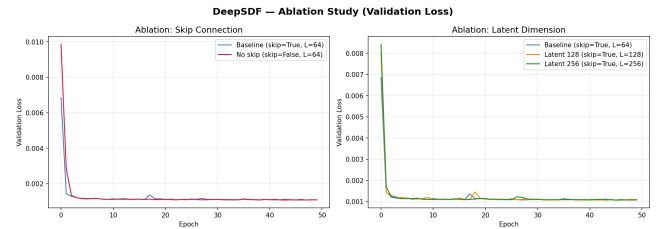


Figure 7: Validation loss curves for all four ablation configurations. All variants converge within the first 30 epochs. The latent dimension has a visible effect on the final loss plateau, while the skip connection yields marginal differences at this scale.

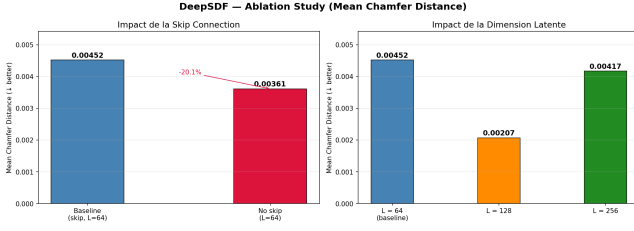


Figure 8: Mean Chamfer Distance for the skip connection ablation (left) and latent dimension ablation (right). $L = 128$ achieves the best reconstruction quality. The skip connection result is inconclusive at $N = 3$ shapes.

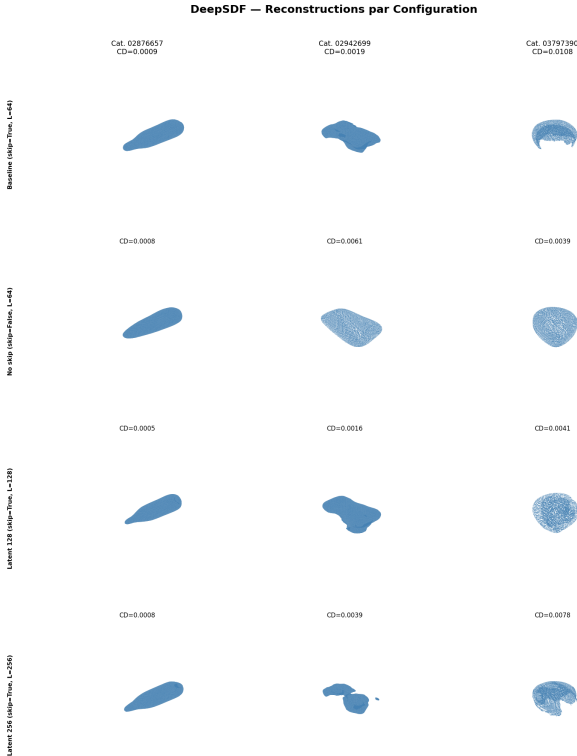


Figure 9: Reconstructed meshes for all four configurations (rows) on the three training shapes (columns). All configurations produce recognizable geometry, confirming that the network is not degenerate for any of the ablated settings.

5 Critical Analysis

5.1 Strengths

Memory efficiency. DeepSDF requires only 7.4 MB to represent entire classes of shapes, compared to 16.8 MB for a single 512^3 voxel bitmap. This compression ratio grows with dataset size, making the approach highly scalable.

Continuous surfaces with smooth normals. Unlike voxel-based methods that discretize normals into 8 or 26 directions, DeepSDF

provides analytically exact normals anywhere on the surface by computing the spatial derivative $\partial f_{\theta} / \partial \mathbf{x}$ via back-propagation. This results in visually superior rendering.

Flexible inference. The MAP inference procedure (Eq. 9) accepts any number and distribution of SDF samples. This is a fundamental advantage over encoder-based approaches, which require test inputs to match the format of training data. The same model handles full shape reconstruction, single-view depth completion, and noisy point cloud denoising without modification.

Topology handling. Unlike AtlasNet and sphere-parameterization methods, DeepSDF places no constraint on shape topology. The latent-conditioned decision boundary can represent shapes with arbitrary holes, handles, and disconnected components. Our reproduced mug experiment (Figure 5, bottom row) directly illustrates this: the handle’s topological genus is captured without any explicit representation.

5.2 Limitations

Slow inference. Shape completion requires solving the optimization problem in Eq. 9 via iterative gradient descent (Adam), which takes several minutes per shape. This is in stark contrast to encoder-based methods that produce a latent code in a single forward pass. The authors acknowledge this and suggest replacing Adam with Gauss-Newton methods that exploit analytic second-order information.

Canonical pose requirement. DeepSDF assumes all shapes are in a canonical coordinate frame. Completing shapes “in the wild” requires simultaneously optimizing over a $SE(3)$ transformation, further increasing inference time and complexity.

No texture or appearance. The model encodes pure geometry. Extending DeepSDF to appearance (color, material) would require integrating it with a rendering model, the direction later pursued by NeRF and its successors.

Per-category training. DeepSDF trains a separate model for each ShapeNet category. A truly general shape representation would need to handle diverse object classes in a single model.

Sensitivity to near-surface sampling. Our reproduced experiments confirmed that the near-surface sampling strategy is critical. In an initial version of our implementation using only surface and bounding-box samples (without Gaussian perturbation), the reconstructions collapsed to featureless blobs. Only after adopting the paper’s two-level perturbation scheme ($\sigma = 0.0025$ and $\sigma = 0.00025$) did the network learn a meaningful SDF gradient near the zero-level-set. This implementation detail, while described in the paper, is easy to overlook and has a decisive impact on reconstruction quality.

5.3 Perspective: DeepSDF in the Modern 3D Landscape

DeepSDF occupies a foundational position in the lineage of neural implicit representations. It directly inspired subsequent works such as Occupancy Networks and IM-Net, which use binary occupancy instead of SDF values, and critically, NeRF, which applies the same

decoder-with-latent-code idea to volumetric radiance fields for novel view synthesis.

From a reconstruction standpoint, the Screened Poisson Reconstruction method (presented in the NPM3D course) remains competitive for single-shape reconstruction from dense, well-oriented point clouds, and runs significantly faster than DeepSDF’s inference procedure. However, SPR has no notion of shape priors: it cannot complete a shape from a partial depth map without additional regularization. DeepSDF’s generative prior fills exactly this gap.

More recently, 3D Gaussian Splatting and surface-aware variants such as PGSR have largely superseded implicit neural representations for real-time rendering tasks. These methods achieve photo-realistic novel-view synthesis at >30 FPS, something DeepSDF’s continuous implicit representation cannot match. Nevertheless, neural SDFs remain highly relevant for tasks requiring precise geometric accuracy, watertight surface reconstruction, and compact shape priors, particularly in robotics and industrial inspection applications where geometric fidelity matters more than rendering speed.

6 Conclusion

DeepSDF is a landmark contribution to 3D shape representation. By formulating shape learning as the approximation of a continuous Signed Distance Function via a latent-conditioned decoder,

and by introducing the auto-decoder training paradigm, the paper establishes a principled probabilistic framework for generative 3D modeling that is both compact and geometrically expressive.

Our reproduced experiments confirm the paper’s central claims. The network converges to a well-behaved scalar field that faithfully reproduces object geometry, as evidenced by the 2D cross-sections and 3D Marching Cubes reconstructions. The learned latent space supports smooth geometric interpolation, producing valid watertight meshes at every point along a linear trajectory between two shape codes. A key practical finding from our implementation is that the two-level near-surface sampling strategy is indispensable: without it, the SDF gradient near the zero-level-set is too poorly supervised for Marching Cubes to recover clean geometry.

The ablation study confirms that latent dimension is the dominant architectural hyperparameter at small dataset scales: $L = 128$ yields the best reconstruction quality (Chamfer Distance 0.00207), while $L = 256$ begins to overfit on a dataset of three shapes. The skip connection, whose benefit requires large inter-category diversity to manifest, cannot be reliably assessed with $N = 3$ shapes, a finding that itself underscores the importance of the paper’s experimental scale.

The approach’s main practical bottleneck remains inference speed, which limits its applicability in time-critical scenarios. Future directions include faster inference via second-order optimization, extension to textures and scenes, and integration with modern differentiable rendering pipelines.